

TwoPointBoundaryProblems

April 2, 2026

1 Métodos de diferencias finitas para problemas de valor de dos puntos

Basado en el trabajo de [Anne Kværnø](#) y [André Massing](#).

Fecha: **25 de marzo de 2025**

1.1 Introducción

En esta nota se presentará el método de diferencias finitas para resolver ecuaciones diferenciales parciales (PDE).

En términos generales, un método de diferencias finitas consta de los siguientes pasos: 1. Discretiza el dominio en el que se define la ecuación. 2. En cada punto de la cuadrícula, reemplace las derivadas con una aproximación, utilizando los valores en los puntos de la cuadrícula vecinos. 3. Reemplazar las soluciones exactas por sus aproximaciones. 4. Resuelve el sistema de ecuaciones resultante.

Primero veremos cómo encontrar aproximaciones a la derivada de una función y luego cómo se pueden usar para resolver problemas de valores en la frontera como

$$\begin{cases} u''(x) + p(x)u'(x) + q(x)u(x) = r(x) & \text{en } (a, b). \\ u(a) = u_a, \quad u(b) = u_b. \end{cases}$$

y ecuaciones diferenciales parciales dependientes del tiempo como la ecuación del calor

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}.$$

Sin embargo, la técnica descrita aquí es aplicable a varias otras EDPs, y, por tanto, es importante intentar comprender la idea subyacente.

1.2 Diferenciación numérica

Esta es la herramienta principal para los métodos de diferencias finitas.

Dada una función u suficientemente suave. ¿Cómo podemos encontrar una aproximación a $u'(x)$ o $u''(x)$ en algún punto dado x , simplemente usando la evaluación de la función en sí?

La derivada de u se define como

$$u'(x) = \lim_{h \rightarrow 0} \frac{u(x+h) - u(x)}{h}.$$

Dado un valor suficientemente pequeño de h , el lado derecho se puede utilizar como una aproximación a la *primera derivada* $u'(x)$. Una pequeña colección de las aproximaciones más utilizadas a la derivada de primer orden $u'(x)$ es

$$u'(x) \approx \begin{cases} \frac{u(x+h) - u(x)}{h} := \partial^+ u(x) & \text{Diferencia hacia adelante.} \\ \frac{u(x) - u(x-h)}{h} := \partial^- u(x) & \text{Diferencia hacia atrás.} \\ \frac{u(x+h) - u(x-h)}{2h} := \partial^\circ u(x) & \text{Diferencia central.} \end{cases}$$

Recuperado de <https://gitlab.com/carlosal1015/partial-differential-equations/-/raw/main/utils/fdm/derivatives.py>

El primero se toma directamente de la definición, al igual que el segundo, y el tercero es solo la media de los dos primeros.

Una aproximación común a la *segunda derivada* es

$$u''(x) \approx \partial^+ \partial^- u(x) = \frac{u(x+h) - 2u(x) + u(x-h)}{h^2}.$$

Ejemplo numérico 1: Probar el método en la función $u(x) = \sin(x)$ en el punto $x = \frac{\pi}{4}$. Comparar con la derivada exacta. Pruebe diferentes tamaños de paso, por ejemplo, $h = 0.1$, $h = 0.01$ y $h = 0.001$. Observe cómo el error en cada caso cambia con h .

```
[1]: import matplotlib.pyplot as plt
import numpy as np

plt.rcParams.update(
    {
        "figure.figsize": (8.0, 4.0),
        "axes.grid": True,
        "lines.markersize": 8,
        "lines.linewidth": 2,
        "font.size": 14,
    }
)
```

```
[2]: # Diferenciación numérica

# Diferencia hacia adelante
def diff_forward(u, x, h=0.1):
    return (u(x + h) - u(x)) / h

# Diferencia hacia atrás
def diff_backward(u, x, h=0.1):
    return (u(x) - u(x - h)) / h
```

```

# Diferencia central para f'(x):
def diff_central(u, x, h=0.1):
    return (u(x + h) - u(x - h)) / (2 * h)

# Diferencia central para f''(x):
def diff2_central(u, x, h=0.1):
    return (u(x + h) - 2 * u(x) + u(x - h)) / h**2

```

```

[3]: # Ejemplo numérico 1
x = np.pi / 4
du_exact = np.cos(x)
ddu_exact = -np.sin(x)
for h in [0.1, 0.01, 0.001]:
    u = np.sin
    du = diff_forward(u, x, h)
    print(f"Approximations to the first derivative for h = {h}")
    print(f"Forward difference: du = {du:12.8f}, Error = {du_exact - du:10.
↵3e}")
    du = diff_backward(u, x, h)
    print(f"Backward difference: du = {du:12.8f}, Error = {du_exact - du:10.
↵3e}")
    du = diff_central(u, x, h)
    print(f"Central difference: du = {du:12.8f}, Error = {du_exact - du:10.
↵3e}")
    print("Approximation to the second derivative")
    ddu = diff2_central(u, x, h)
    print(f"Central difference: ddu = {ddu:12.8f}, Error = {ddu_exact - ddu:10.
↵3e}")

```

```

Approximations to the first derivative for h = 0.1
Forward difference: du = 0.67060297, Error = 3.650e-02
Backward difference: du = 0.74125475, Error = -3.415e-02
Central difference: du = 0.70592886, Error = 1.178e-03
Approximation to the second derivative
Central difference: ddu = -0.70651772, Error = -5.891e-04
Approximations to the first derivative for h = 0.01
Forward difference: du = 0.70355949, Error = 3.547e-03
Backward difference: du = 0.71063050, Error = -3.524e-03
Central difference: du = 0.70709500, Error = 1.179e-05
Approximation to the second derivative
Central difference: ddu = -0.70710089, Error = -5.893e-06
Approximations to the first derivative for h = 0.001
Forward difference: du = 0.70675311, Error = 3.537e-04
Backward difference: du = 0.70746022, Error = -3.534e-04
Central difference: du = 0.70710666, Error = 1.179e-07

```

Approximation to the second derivative

Central difference: ddu = -0.70710672, Error = -5.901e-08

1.3 Análisis de errores

En este caso, el análisis del error es bastante simple: realice una expansión de Taylor del error alrededor de x . La expansión de Taylor se convierte en una serie de potencias en h .

La expansión para el error de la diferencia directa es

$$\begin{aligned}e(x; h) &= u'(x) - \frac{u(x+h) - u(x)}{h} \\&= u'(x) - \frac{[u(x) + u'(x)h + \frac{1}{2}u''(\xi)h^2] - u(x)}{h} \\&= -\frac{1}{2}u''(\xi)h, \quad \text{para algún } \xi \in (x, x+h).\end{aligned}$$

La expansión del error de la diferencia central es un poco más complicada

$$\begin{aligned}e(x; h) &= u'(x) - \frac{u(x+h) - u(x-h)}{2h} \\&= u'(x) - \frac{[u(x) + u'(x)h + \frac{1}{2}u''(x)h^2 + \frac{1}{6}u'''(\xi_1)h^3] - [u(x) - u'(x)h + \frac{1}{2}u''(x)h^2 - \frac{1}{6}u'''(\xi_2)h^3]}{2h} \\&= -\frac{1}{12}[u'''(\xi_1) + u'''(\xi_2)]h^2 \\&= -\frac{1}{6}u'''(\eta)h^2, \quad \text{para algún } \eta \in (x-h, x+h).\end{aligned}$$

En el último paso, los dos términos restantes se combinaron mediante el teorema del valor intermedio. El error de aproximación de la derivada de segundo orden se puede encontrar de manera similar.

Problema 1: Utilice la expansión de Taylor para demostrar que $\forall u \in C^4([a, b]): |\partial^+ \partial^- u(x) - u''(x)| = \mathcal{O}(h^2)$.

Recordatorio. El orden de una aproximación es p si y solamente si existe una constante C independiente de h tal que $\forall h > 0: |e(h; x)| \leq Ch^p$.

En la práctica, es suficiente demostrar que la expansión de potencia del error satisface

$$e(x, h) = C_p h^p + C_{p+1} h^{p+1} + \dots, \text{ con } C_p \neq 0.$$

Las aproximaciones hacia adelante y hacia atrás son de orden 1, las diferencias centrales de orden 2.

Usaremos mucho estas fórmulas en la secuela, así que resumamos los resultados, incluidos los términos de error.

Fórmulas de diferencias para derivadas:

$$\begin{aligned}
u'(x) &= \begin{cases} \underbrace{\frac{u(x+h) - u(x)}{h}}_{\partial^+ u(x)} - \frac{h}{2} u''(\xi), & \text{Diferencia hacia adelante} \\ \underbrace{\frac{u(x) - u(x-h)}{h}}_{\partial^- u(x)} + \frac{h}{2} u''(\xi), & \text{Diferencia hacia atrás} \\ \underbrace{\frac{u(x+h) - u(x-h)}{2h}}_{\partial^0 u(x)} - \frac{h^2}{6} u'''(\xi). & \text{Diferencia central} \end{cases} \\
u''(x) &= \underbrace{\frac{u(x+h) - 2u(x) + u(x-h)}{h^2}}_{\partial^+ \partial^- u(x)} - \frac{h^2}{12} u^{(4)}(\xi), \quad \text{Diferencia central}
\end{aligned}$$

1.4 Método de diferencias finitas para el problema de Poisson 1d

Comenzamos nuestro viaje con el siguiente problema. Sea $[a, b] \subset \mathbb{R}$ un intervalo finito, junto con una función $f: [a, b] \rightarrow \mathbb{R}$. Entonces, el *problema de valor de frontera de dos puntos* es encontrar un $u: [a, b] \rightarrow \mathbb{R}$ tal que

$$-u''(x) = f(x) \quad \forall x \in (a, b), \quad (1a)$$

$$u(a) = u_a, \quad u(b) = u_b \quad (1b)$$

donde u_a y u_b reciben valores dados. *Para el resto de esta sección asumiremos que $a = 0$, $b = 1$.*

Tenga en cuenta que estas son ecuaciones diferenciales ordinarias de **segundo orden** y, por lo tanto, requieren dos valores de suma para determinar una solución completa, uno en cada punto frontera.

Este es el contraste del problema de valor inicial de segundo orden

$$\begin{aligned}
u'' &= f(t, u(t), u'(t)) \quad \forall t \in (0, 1) \\
u(0) &= u_0, \\
u'(0) &= u_1
\end{aligned}$$

donde tenemos dos condiciones en el punto final izquierdo. La razón principal es que en el primer caso, x es una variable espacial, mientras que en el último caso, t es una variable de tiempo.

En lugar de intentar calcular $u(x)$ exactamente, ahora intentaremos calcular una aproximación numérica u_Δ de $u(x)$. Como muchas veces antes, comenzamos definiendo $n+1$ puntos equidistantes $\{x_i\}_{i=0}^n$ con un tamaño de malla $h = \frac{b-a}{n}$ de modo que

$$x_i = a + ih \quad \text{para } i = 0, 1, \dots, n.$$

Entonces la derivada de segundo orden u'' puede aproximarse mediante el **operador de diferencia central** definido por

$$\partial^+ \partial^- u(x_i) = \frac{u(x_i + h) - 2u(x_i) + u(x_i - h)}{h^2} \approx u''(x_i) = -f(x_i). \quad (1)$$

Now the idea is to numerically solve for the two-point boundary value problem by associating an unknown variable $U_i = u_\Delta(x_i)$ to every grid point x_i for $i = 0, \dots, N$. Of course, the goal is to find an $U_i \approx u(x_i)$. Keeping in mind that $-u'' = f$, we demand that at the **internal** grid points $\{x_i\}_{i=1}^N$ the unknowns satisfy

$$-\partial^+ \partial^- U_i = -\frac{U_{i+1} - 2U_i + U_{i-1}}{h^2} = f(x_i) \text{ para } i = 1, \dots, N-1. \quad (2)$$

Tenga en cuenta que en x_0 y x_N , $\partial^+ \partial^- U_i$ no está bien definido ya que no tenemos un punto a la izquierda (respectivamente a la derecha) de x_0 (respectivamente x_N). Esto conduce al sistema lineal *no cuadrado* $N-1 \times N+1$ de la forma

$$\frac{1}{h^2} \underbrace{\begin{bmatrix} -1 & 2 & -1 & & & \\ & -1 & 2 & -1 & & \\ & & -1 & 2 & -1 & \\ & & & \ddots & \ddots & \ddots \\ & & & & -1 & 2 & -1 \end{bmatrix}}_{\tilde{A}} \underbrace{\begin{bmatrix} U_0 \\ U_1 \\ U_2 \\ U_3 \\ \vdots \\ U_N \end{bmatrix}}_{\tilde{U}} = \underbrace{\begin{bmatrix} f(x_1) \\ f(x_2) \\ f(x_3) \\ \vdots \\ f(x_{N-1}) \end{bmatrix}}_{\tilde{F}} \quad (3)$$

Por reflejo, inmediatamente preguntamos si este sistema *siempre tiene solución* y tiene una *solución única*. Desafortunadamente, la matriz $\tilde{A}$ anterior no es cuadrada y tiene más columnas que filas y, por lo tanto, tiene un núcleo no trivial. Un ejemplo es $U_0 = U_1 = U_2 = \dots = U_{N-1} = U_N = c$ para cualquier constante c . ¡Ahí es donde entran en juego las condiciones de frontera!

Entonces, para cerrar el sistema, incorporemos la condición de frontera estableciendo

$$U_0 = u_a, \quad U_N = u_b.$$

These trivial equations can be added to the system above, leading to final problem:

Encuentre $U = [U_0, \dots, U_N] \in \mathbb{R}^{N+1}$ tal que

$$\frac{1}{h^2} \underbrace{\begin{bmatrix} h^2 & & & & & \\ -1 & 2 & -1 & & & \\ & -1 & 2 & -1 & & \\ & & -1 & 2 & -1 & \\ & & & \ddots & \ddots & \ddots \\ & & & & -1 & 2 & -1 \\ & & & & & h^2 \end{bmatrix}}_A \underbrace{\begin{bmatrix} U_0 \\ U_1 \\ U_2 \\ U_3 \\ \vdots \\ U_{N-1} \\ U_N \end{bmatrix}}_U = \underbrace{\begin{bmatrix} u_a \\ f(x_1) \\ f(x_2) \\ f(x_3) \\ \vdots \\ f(x_{N-1}) \\ u_b \end{bmatrix}}_F \quad (4)$$

1.5 Implementación

```
[4]: def fdm_poisson1d_matrix(N):
```

"""Calcula la matriz de diferencias finitas para el problema de Poisson en 1D

Parámetros:

N (int): Número de puntos de la malla $\{x_i\}_{i=0}^N$ contando desde 0.

Returns:

A (ndarray): Matriz de diferencias finitas

"""

Tamaño de la malla

$h = 1.0 / N$

Defina la matriz cero A del tamaño correcto

$A = \text{np.zeros}((N + 1, N + 1))$

Defina la parte tridiagonal de A

$hh = h * h$

for i in range(1, N):

$A[i, i - 1] = -1 / hh$

$A[i, i] = 2 / hh$

$A[i, i + 1] = -1 / hh$

Establezca $a_{00} = a_{NN} = 1$ para incorporar la condición de frontera

$A[0, 0] = 1$

$A[N, N] = 1$

return A

def apply_bcs(F , bcs):

"Incorporar la condición de frontera en el vector rhs"

$F[0], F[-1] = \text{bcs}[0], \text{bcs}[1]$

[5]: # Solución de referencia analítica

def $u(x)$:

return $\text{np.sin}(2 * \text{np.pi} * x)$

def $f(x)$:

return $(2 * \text{np.pi}) ** 2 * \text{np.sin}(2 * \text{np.pi} * x)$

$N = 10$

Defina puntos de la malla

```
x = np.linspace(0, 1, N + 1)
xfine = np.linspace(0, 1, 10 * N)
```

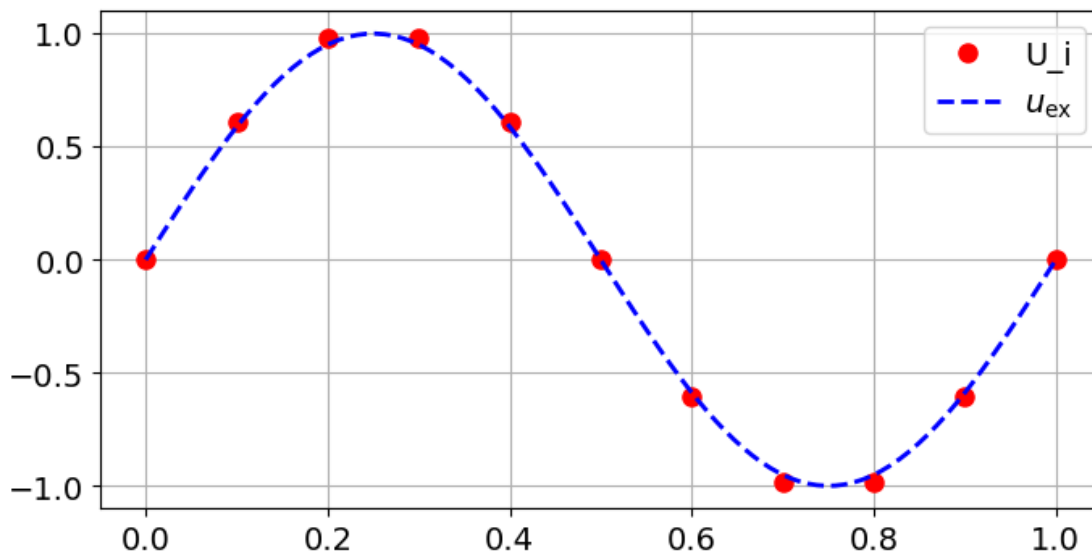
```
# Calcular A y F
A = fdm_poisson1d_matrix(N)
F = f(x)
print(A)
print(F)
```

```
[[ 1.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.]
 [-100. 200. -100.  0.  0.  0.  0.  0.  0.  0.  0.]
 [ 0. -100. 200. -100.  0.  0.  0.  0.  0.  0.  0.]
 [ 0.  0. -100. 200. -100.  0.  0.  0.  0.  0.  0.]
 [ 0.  0.  0. -100. 200. -100.  0.  0.  0.  0.  0.]
 [ 0.  0.  0.  0. -100. 200. -100.  0.  0.  0.  0.]
 [ 0.  0.  0.  0.  0. -100. 200. -100.  0.  0.  0.]
 [ 0.  0.  0.  0.  0.  0. -100. 200. -100.  0.  0.]
 [ 0.  0.  0.  0.  0.  0.  0. -100. 200. -100.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  0. -100. 200. -100.]
 [ 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  1.]]
[ 0.00000000e+00  2.32048317e+01  3.75462063e+01  3.75462063e+01
  2.32048317e+01  4.83471178e-15 -2.32048317e+01 -3.75462063e+01
 -3.75462063e+01 -2.32048317e+01 -9.66942355e-15]
```

```
[6]: # Incorporar condiciones de frontera
bcs = [u(0), u(1)]
apply_bcs(F, bcs)

# Resolver AU = F
# (Introduciremos un solucionador disperso cuando analicemos problemas 2D.)
U = np.linalg.solve(A, F)
```

```
[7]: # Grafique una solución discreta en la malla de discretización elegida
plt.plot(x, U, "or", label="U_i")
plt.plot(xfine, u(xfine), "--b", label="$u_{\mathrm{ex}}$")
plt.legend()
plt.show()
```

Problema 2

Aplique el método de **soluciones manufacturadas**: tome una función conocida $u(x)$ y calcule los datos correspondientes f y u_a, u_b tales que u es la solución al problema de la frontera de dos puntos (1a)–(1b)

Por ejemplo, puede elegir $u(x) = \cos(2\pi x)$. ¿Qué es entonces $f = -u''$, $u_a = u(0)$, $u_b = u(1)$?

Después de haber elegido su solución manufacturada favorita (suave), puede calcular el error

$$\text{err}(h) = \text{err}(N) = \max_{i \in \{0, \dots, N\}} |U_i - u(x_i)|$$

en función de $h = \frac{1}{N}$.

Para $N_1 = 5$, $N_2 = 10$, $N_3 = 20$, $N_4 = 40$, $N_5 = 80$. * Calcule y grafique la solución numérica y la solución analítica. * Registre el resultado $\text{err}(N)$.

Al final de su experimento, calcule el **orden de convergencia experimental** para cada nivel de refinamiento $N_2, \dots, N_5$ definido

$$\text{EOC}(N_i) = \frac{\log(\text{err}(N_i)) - \log(\text{err}(N_{i-1}))}{\log(N_{i-1}) - \log(N_i)}.$$

¿Qué orden de convergencia espera y cuál obtiene?

1.6 General two point boundary problems (BVP)

A continuación, discutiremos la solución numérica de un problema de valor de frontera de dos puntos de la forma

$$u'' + p(x)u' + q(x)u = r(x), \quad a \leq x \leq b, \quad u(a) = u_a, \quad u(b) = u_b,$$

donde p, q son funciones de x dadas y los valores de frontera u_a y u_b son constantes dadas.

Como antes, se construye un método de diferencias finitas para este problema con los siguientes pasos:

Paso 1: Dado el intervalo $[a, b]$. Elija algunos $N \in \mathbb{N}$, defina el tamaño de paso $h = \frac{b-a}{N}$ y los puntos de la malla $x_i = a + ih$, $i = 0, 1, \dots, N$.

Paso 2: Para cada punto interior de la malla x_i , $i = 1, \dots, N-1$, reemplace las derivadas por sus aproximaciones en el problema de valor de frontera. El resultado es

$$\frac{u(x_i + h) - 2u(x_i) + u(x_i - h))}{h^2} + p(x_i) \frac{u(x_i + h) - u(x_i - h)}{2h} + q(x_i)u(x_i) + \mathcal{O}(h^2) = r(x_i)$$

para cada $i = 1, 2, \dots, N-1$, y el término $\mathcal{O}(h^2)$ representa los errores en las fórmulas de diferencia.

Paso 3: Ignore el término de error y reemplace la solución exacta $u(x_i)$ por su aproximación numérica (y aún desconocida) U_i

$$\frac{U_{i+1} - 2U_i + U_{i-1}}{h^2} + p(x_i) \frac{U_{i+1} - U_{i-1}}{2h} + q(x_i)U_i = r(x_i), \quad i = 1, \dots, N-1.$$

Esta es la *discretización* del problema de valor de frontera.

Si ahora incluimos los dos valores de frontera como ecuaciones

$$U_0 = u_a \text{ y } U_N = u_b,$$

la discretización es un sistema lineal de ecuaciones

$$AU = b,$$

donde $A \in \mathbb{R}^{(N+1) \times (N+1)}$ es una matriz y $U = [U_0, \dots, U_N]^T$. O más específico, al multiplicar las ecuaciones por h^2 terminamos con:

$$A = \begin{bmatrix} 1 & 0 & & & & & & \\ v_1 & d_1 & w_1 & & & & & \\ & v_2 & d_2 & w_2 & & & & \\ & & v_3 & \ddots & \ddots & & & \\ & & & \ddots & \ddots & w_{N-2} & & \\ & & & & v_{N-1} & d_{N-1} & w_{N-1} & \\ & & & & & 0 & 1 & \end{bmatrix} \quad \text{con} \quad \begin{cases} v_i = 1 - \frac{h}{2}p(x_i) \\ d_i = -2 + h^2q(x_i) \\ w_i = 1 + \frac{h}{2}p(x_i) \end{cases}.$$

El lado derecho b está dado por

$$b = [u_a, h^2r(x_1), \dots, h^2r(x_{N-1}), u_b]^T.$$

Obviamente, la primera y la última ecuaciones son triviales de resolver y, por lo tanto, a menudo se incluyen en el lado derecho.

Paso 4: Resuelva $AU = b$ con respecto a U .

Ejemplo 1: Consideremos la ecuación

$$u'' + 2u' - 3u = 9x, \quad u(0) = u_a = 1, \quad u(1) = u_b = e^{-3} + 2e - 5 \approx 0.486351,$$

con solución exacta $u(x) = e^{-3x} + 2e^x - 3x - 2$.

Elija N , sea $h = \frac{1}{N}$. Utilice las diferencias centrales para u' y u'' , tales que

$$\frac{u(x_i + h) - 2u(x_i) + u(x_i - h))}{h^2} + 2\frac{u(x_i + h) - u(x_i - h)}{2h} - 3u(x_i) + \mathcal{O}(h^2) = 9x_i, \quad i = 1, \dots, N.$$

Sea $U_i \approx u(x_i)$. Multiplica por h^2 en ambos lados, incluye $U_0 = u_a$ o $U_N = u_b$ y limpie el desorden:

$$\begin{aligned} U_0 &= 1, \\ (1 - h)U_{i-1} + (-2 - 3h^2)U_i + (1 + h)U_{i+1} &= 9x_i h^2, \quad i = 1, \dots, N - 1, \\ U_N &= 0.486351. \end{aligned}$$

Para ser aún más concreto, para $N = 4$, obtenemos $h = 0.25$. El sistema lineal de ecuaciones se convierte en

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0.75 & -2.1875 & 1.25 & 0 & 0 \\ 0 & 0.75 & -2.1875 & 1.25 & 0 \\ 0 & 0 & 0.75 & -2.1875 & 1.25 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} U_0 \\ U_1 \\ U_2 \\ U_3 \\ U_4 \end{bmatrix} = \begin{bmatrix} 1. \\ 0.140625 \\ 0.28125 \\ 0.421875 \\ 0.48635073 \end{bmatrix}.$$

La primera y la última ecuación son triviales de resolver, por lo que en la práctica tienes un sistema de 3 ecuaciones con 3 incógnitas,

$$\begin{bmatrix} -2.1875 & 1.25 & 0 \\ 0.75 & -2.1875 & 1.25 \\ 0 & 0.75 & -2.1875 \end{bmatrix} \begin{bmatrix} U_1 \\ U_2 \\ U_3 \end{bmatrix} = \begin{bmatrix} 0.140625 - 0.75 \cdot 1 \\ 0.28125 \\ 0.421875 - 1.25 \cdot 0.48635073 \end{bmatrix},$$

con la solución

$$U_1 \approx 0.293176, \quad U_2 \approx 0.025557, \quad U_3 \approx 0.093820.$$

A modo de comparación, la solución analítica en estos puntos es

$$u(0.25) \approx 0.290417, \quad u(0.5) \approx 0.020573, \quad u(0.75) \approx 0.089400.$$

1.7 Implementación

Para simplificar, la implementación debajo solo se realiza para problemas de valor de frontera con coeficientes constantes, es decir, $p(x) = p$ y $q(x) = q$. Esto hace que las diagonales, subdiagonales y superdiagonales sean constantes, excepto en la primera y la última fila. Se incluye una función adicional para construir matrices *tridiagonales*, es decir, matrices donde todas las entradas fuera de la diagonal, subdiagonal y superdiagonal son iguales a cero.

La implementación consiste en 1. Elija N , sea $h = \frac{b-a}{N}$ y $x_i = a + ih$, $i = 0, \dots, N$. 2. Construya la matriz $A \in \mathbb{R}^{(N+1) \times (N+1)}$ y el vector $b \in \mathbb{R}^{N+1}$. La matriz A es tridiagonal y, excepto en la primera y última fila, tiene los elementos $v = 1 - \frac{h}{2}p$ debajo de la diagonal, $d = -2 + h^2q$ como elementos diagonales y $w = 1 + \frac{h}{2}p$ encima de la diagonal. 3. Construya el vector $b = [b_0, \dots, b_N]^T$ con elementos $b_i = h^2 r(x_i)$ para $i = 1, \dots, N - 1$. 4. Modifique la primera y última fila de la matriz A y el primer y último elemento del vector b , dependiendo de las condiciones de frontera. 5. Resuelve el sistema $AU = b$.

```
[8]: def tridiag(v, d, w, N):
      """
      Función de ayuda
      Devuelve una matriz tridiagonal A=tridiag(v, d, w) de dimensión N x N.
      """

      e = np.ones(N) # array [1,1,...,1] of length N
      A = v * np.diag(e[1:], -1) + d * np.diag(e) + w * np.diag(e[1:], 1)
      return A
```

```
[9]: np.diag?
```

```
Signature:      np.diag(v, k=0)
Call signature: np.diag(*args, **kwargs)
Type:          _ArrayFunctionDispatcher
String form:    <function diag at 0x7f00e83024b0>
File:          /usr/lib/python3.14/site-
packages/numpy/lib/_twodim_base_impl.py
Docstring:
Extract a diagonal or construct a diagonal array.
```

See the more detailed documentation for ``numpy.diagonal`` if you use this function to extract a diagonal and wish to write to the resulting array; whether it returns a copy or a view depends on what version of numpy you are using.

Parameters

v : array_like
 If `v` is a 2-D array, return a copy of its `k`-th diagonal.
 If `v` is a 1-D array, return a 2-D array with `v` on the `k`-th diagonal.

k : int, optional
 Diagonal in question. The default is 0. Use `k>0` for diagonals above the main diagonal, and `k<0` for diagonals below the main diagonal.

Returns

out : ndarray
 The extracted diagonal or constructed diagonal array.

See Also

diagonal : Return specified diagonals.
diagflat : Create a 2-D array with the flattened input as a diagonal.
trace : Sum along diagonals.
triu : Upper triangle of an array.

tril : Lower triangle of an array.

Examples

```
>>> import numpy as np
>>> x = np.arange(9).reshape((3,3))
>>> x
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
```

```
>>> np.diag(x)
array([0, 4, 8])
>>> np.diag(x, k=1)
array([1, 5])
>>> np.diag(x, k=-1)
array([3, 7])
```

```
>>> np.diag(np.diag(x))
array([[0, 0, 0],
       [0, 4, 0],
       [0, 0, 8]])
```

Class docstring:

Class to wrap functions with checks for `__array_function__` overrides.

All arguments are required, and can only be passed by position.

Parameters

dispatcher : function or None

The dispatcher function that returns a single sequence-like object of all arguments relevant. It must have the same signature (except the default values) as the actual implementation.

If `None`, this is a `like` dispatcher and the `_ArrayFunctionDispatcher` must be called with `like` as the first (additional and positional) argument.

implementation : function

Function that implements the operation on NumPy arrays without overrides. Arguments passed calling the `_ArrayFunctionDispatcher` will be forwarded to this (and the `dispatcher`) as if using `*args, **kwargs`.

Attributes

`_implementation` : function

The original implementation passed in.

```

[10]: # Ejemplo 1, problema de valor de frontera

# Definir la ecuación
#  $u^{\prime\prime} + p u^{\prime} + q u = r(x)$  en el intervalo  $[a, b]$ 
# Condición de frontera:  $u(a) = u_a$  y  $u(b) = u_b$ 

p = 2
q = -3

def r(x):
    return 9 * x

a, b = 0, 1
ua, ub = 1, np.exp(-3) + 2 * np.exp(1) - 5

# La solución exacta (si se conoce)
def u_eksakt(x):
    return np.exp(-3 * x) + 2 * np.exp(x) - 3 * x - 2

# Configurar el sistema discreto
N = 4 # Número de intervalos

# Iniciar la discretización
h = (b - a) / N # Stepsize
x = np.linspace(a, b, N + 1) # Los puntos de la malla  $x_0=a, x_1=a+h, \dots, x_N=b$ 

# Haga un borrador de la matriz A (la primera y la última fila deben ajustarse)
v = 1 - 0.5 * h * p # Subdiagonal
d = -2 + h**2 * q # Diagonal
w = 1 + 0.5 * h * p # Superdiagonal
A = tridiag(v, d, w, N + 1)

# Haga un borrador del vector b
b = h**2 * r(x)

# Modificar la primera ecuación (frontera izquierda)
A[0, 0] = 1
A[0, 1] = 0
b[0] = ua

# Modificar la última ecuación (frontera derecha)

```

```

A[N, N] = 1
A[N, N - 1] = 0
b[N] = ub

U = np.linalg.solve(A, b) # Resuelve la ecuación

```

Para verificar los cálculos realizados anteriormente, imprima la matriz A , el vector b y la solución numérica U .

```

[11]: # Imprime la matriz A, el lado derecho b la solución numérica y exacta
print("A =\n", A)
print("\nb =\n ", b)
print("\nU =\n ", U)
print("\nu(x)=\n", u_eksakt(x))

```

```

A =
[[ 1.      0.      0.      0.      0.   ]
 [ 0.75   -2.1875  1.25     0.      0.   ]
 [ 0.      0.75   -2.1875  1.25     0.   ]
 [ 0.      0.      0.75   -2.1875  1.25  ]
 [ 0.      0.      0.      0.      1.   ]]

b =
[1.      0.140625  0.28125  0.421875  0.48635073]

U =
[1.      0.29317568 0.02555744 0.09382011 0.48635073]

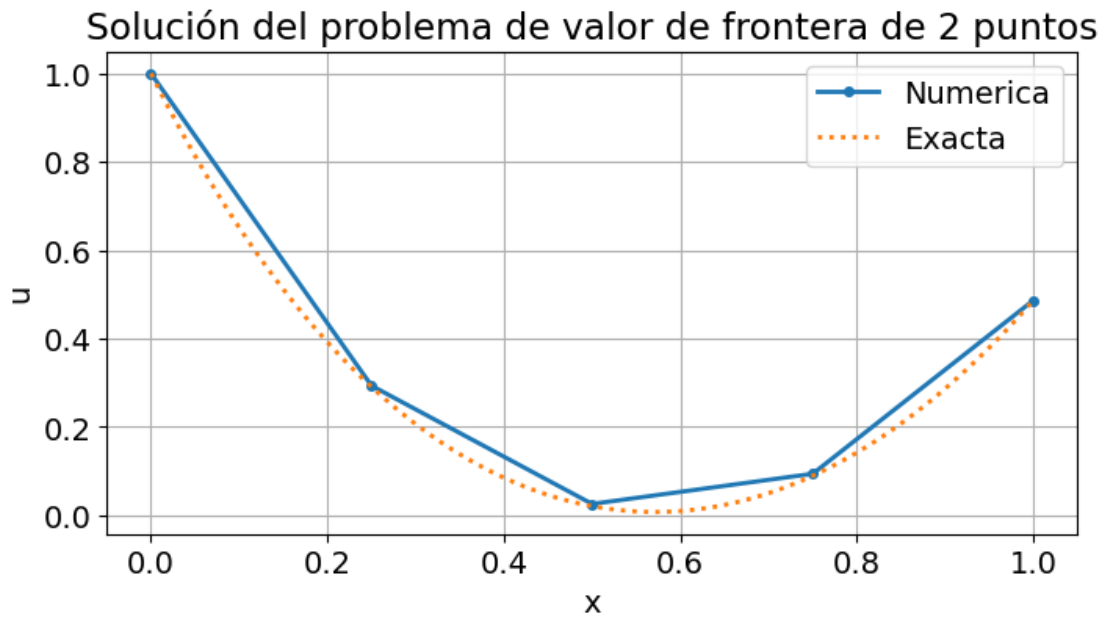
u(x)=
[1.      0.29041739 0.0205727  0.08939926 0.48635073]

```

```

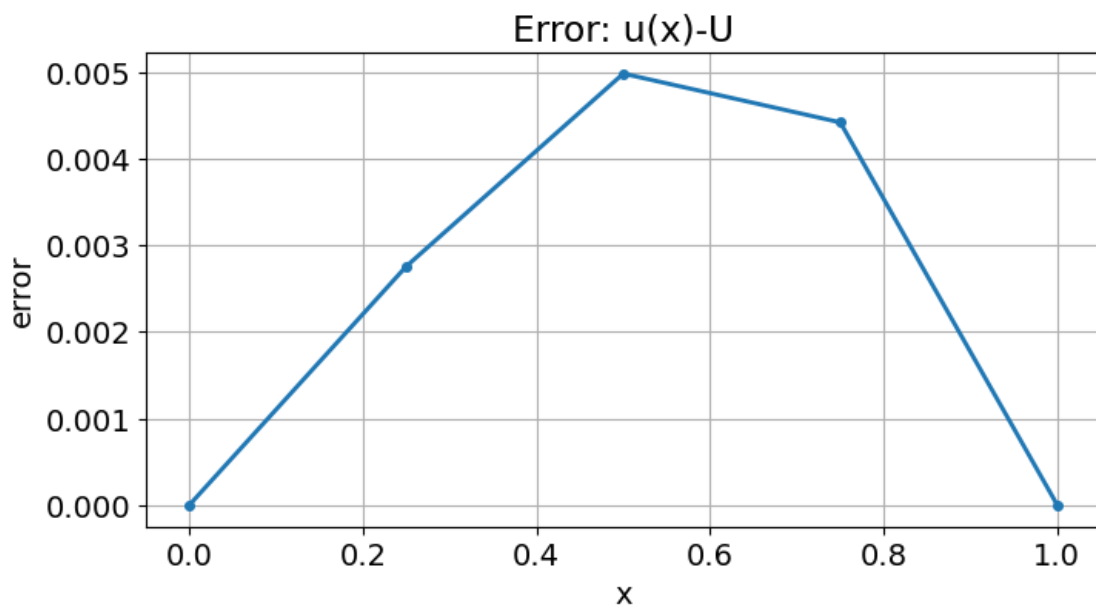
[12]: # Grafique la solución del problema de valor de frontera.
xe = np.linspace(0, 1, 101)
plt.plot(x, U, "-.")
plt.plot(xe, u_eksakt(xe), ":")
plt.xlabel("x")
plt.ylabel("u")
plt.legend(["Numerica", "Exacta"])
plt.title("Solución del problema de valor de frontera de 2 puntos");

```



```
[13]: # Grafica el error |u(x)-U| en los puntos de la malla
error = np.abs(u_eksakt(x) - U)
plt.plot(x, error, "-.")
plt.xlabel("x")
plt.ylabel("error")
plt.title("Error: u(x)-U")
print("Max error = {:.3e}".format(max(abs(error))))
```

Max error = 4.985e-03



1.8 Condiciones de frontera

Para obtener una solución única de un problema de valor de frontera (o una EDP), se debe conocer cierta información sobre la solución, generalmente proporcionada en la frontera.

Las condiciones de frontera más comunes son: 1. Condición de Dirichlet: La solución se conoce en la frontera. 2. Condición de Neumann: La derivada se conoce en la frontera. 3. Condición de Robin (o mixta): Una combinación de estos dos últimos.

En el ejemplo anterior, se utilizaron las condiciones de Dirichlet. Ahora veremos cómo manejar las condiciones de Neumann. Las condiciones de Robin se pueden tratar de manera similar.

Dado el problema de valor de frontera con una condición de Neumann en la frontera izquierda:

$$u'' + p(x)u' + q(x)u = r(x), \quad a \leq x \leq b, \quad u'(a) = u'_a, \quad u(b) = u_b.$$

Aquí, u'_a es un valor dado. En este caso, la solución $u(a)$ y su correspondiente aproximación U_0 son desconocidas y necesitamos alguna fórmula de diferencia también para el punto $a = x_0$. La opción más sencilla es utilizar una diferencia hacia adelante.

$$u'_a = \frac{u(x_1) - u(x_0)}{h} + \mathcal{O}(h) \text{ resulting in } \frac{U_1 - U_0}{h} = u'_a.$$

Pero esto es sólo una aproximación de primer orden y, por lo tanto, se debe tener una precisión menor esperado. También podríamos usar una aproximación de segundo orden usando los valores en los puntos de la cuadrícula x_0 , x_1 y x_2 , pero esto arruinará la bonita tridiagonal estructura de la matriz de coeficientes.

En su lugar, utilice la idea de una *frontera falsa*:

Supongamos que la solución se puede extender fuera de la frontera $x = a$, hasta llegar a un punto de la malla ficticia $x_{-1} = a - h$, donde también asumimos que hay una aproximación aproximada e igualmente ficticia U_{-1} a $u(x_{-1})$.

Entonces tenemos dos fórmulas de diferencia en el punto a , una para el problema de valor de frontera en sí y una diferencia central para las condiciones de frontera:

$$\begin{aligned} \frac{U_1 - 2U_0 + U_{-1}}{h^2} + p(x_0)\frac{U_1 - U_{-1}}{2h} + q(x_0)U_0 &= r(x_0), \\ \frac{U_1 - U_{-1}}{2h} &= u'_a. \end{aligned}$$

Ahora podemos resolver la segunda ecuación con respecto a U_{-1} e insertar la solución en la primera ecuación. Esto produce la nueva ecuación

$$\frac{2U_1 - 2U_0 - 2hu'_a}{h^2} + p(x_0)u'_a + q(x_0)U_0 = r(x_0).$$

Entonces lo único que ha cambiado es la primera ecuación. Y dado que se han utilizado diferencias centrales tanto para el problema de valor de frontera como para la condición de frontera, se puede demostrar que el orden general de esta aproximación es 2.

Ejemplo 2:

Consideramos el mismo ejemplo que antes, pero ahora con una condición de Neumann en la frontera izquierda.

$$u'' + 2u' - 3u = 9x, \quad u'(0) = u'_a = -4, \quad u(1) = u_b = -2e^{-3} + e - 5 \approx 0.48635073,$$

que tiene la solución analítica $u(x) = e^{-3x} - 2e^x - 3x - 2$. La ecuación en diferencias modificada en la frontera $x_0 = 0$ es

$$\frac{2U_1 - 2U_0 - 2u'_a h}{h^2} + 2u'_a - 3U_0 = 0.$$

Multiplicamos esta ecuación por h^2 e incluimos la ecuación como discretización para el punto de la malla x_0 . Con esto obtenemos el sistema

$$\begin{aligned} (-2 - 3h^2)U_0 - 2U_1 &= (2h - 2h^2)u'_a, \\ (1 - h)U_{i-1} + (-2 - 3h^2)U_i + (1 + h)U_{i+1} &= 9h^2x_i, \quad i = 1, \dots, N-1, \\ U_N &= u_b, \end{aligned}$$

que, para $N = 4$ y $h = 0.25$ se convierte en

$$\begin{bmatrix} -2.1875 & 2 & 0 & 0 & 0 \\ 0.75 & -2.1875 & 1.25 & 0 & 0 \\ 0 & 0.75 & -2.1875 & 1.25 & 0 \\ 0 & 0 & 0.75 & -2.1875 & 1.25 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} U_0 \\ U_1 \\ U_2 \\ U_3 \\ U_4 \end{bmatrix} = \begin{bmatrix} -1.5 \\ 0.140625 \\ 0.28125 \\ 0.421875 \\ 0.48635073 \end{bmatrix}.$$

La solución del sistema lineal es

$$U_0 \approx 0.92103219, \quad U_1 \approx 0.25737896, \quad U_2 \approx 0.01029386, \quad U_3 \approx 0.08858688.$$

Problema 3:

Consideremos ahora el problema de valor de frontera de dos puntos del Ejemplo 2; eso es,

$$u'' + 2u' - 3u = 9x, \quad u'(0) = u'_a = -4, \quad u(1) = u_b = -2e^{-3} + e - 5$$

que tiene la solución analítica $u(x) = e^{-3x} - 2e^x - 3x - 2$.

Aquí queremos que pruebe las dos posibles implementaciones de las condiciones de frontera de Neumann discutidas anteriormente.

a) Modifique la matriz A y el lado derecho b para implementar la condición de frontera de Neumann $u'(0) = u'_a = -4$ a través de la aproximación $u'(a) \approx \frac{U_1 - U_0}{h} = u'_a$. Ahora, utilizando la solución analítica, ejecute un estudio de convergencia en el Problema 2. Grafique/escriba la tabla del orden de convergencia experimental resultante. ¿Qué orden de convergencia obtienes?

b) Ahora modifique la matriz A y el lado derecho b para implementar la condición de frontera de Neumann usando la idea de una frontera falsa como se describió anteriormente. Rehaga el estudio de convergencia, grafique/escriba la tabla del orden de convergencia experimental resultante. ¿Qué orden de convergencia obtienes esta vez?